



**Politecnico
di Torino**

Politecnico di Torino

System and Device Programming

A.a. 2024/2025

C2 Project - Shell

Supervisor:

Gianpiero Cabodi

Candidates:

Riccardo Scanu

Marco Piasso

Milica Dimitrievska

Table of Contents

List of Figures	IV
1 Introduction	1
1.1 Summary	1
1.2 Environment	1
1.3 Traps and System Calls	2
1.4 Problem Definition	3
1.5 System Call Concurrency and File Descriptors	4
1.6 Process Management System Calls	4
1.7 Testing	5
1.8 Project Structure	5
2 Kernel Configuration	7
2.1 Configuration File	7
2.2 System Call Dispatcher	8
2.2.1 Role of the Trapframe	8
2.2.2 Dispatching System Calls	9
2.2.3 Error Handling and Return Conventions	10
2.2.4 Entering a Forked Child Process	10
2.2.5 Concurrency of System Calls	10
2.2.6 Atomicity of Read and Write Operations	11
3 File Handling	13
3.1 Supporting Structures and Functions	13
3.1.1 The <code>openfile</code> Structure	13
3.1.2 System and Process File Tables	14
3.1.3 Utility Functions and Macros	14
3.1.4 Defined Constants	15
3.2 System Calls Implementation	15
3.2.1 System Call: <code>sys_open</code>	16
3.2.2 Closing Files: <code>sys_close</code>	18

3.2.3	Reading from Files: <code>sys_read</code>	19
3.2.4	Writing into Files: <code>sys_write</code>	21
3.2.5	Manipulating File Offsets: <code>sys_lseek</code>	23
3.2.6	Duplicating File Descriptors: <code>sys_dup2</code>	25
3.2.7	Change Current Directory: <code>sys_chdir</code>	27
3.2.8	Name of Current Working Directory: <code>sys_getcwd</code>	28
4	Virtual Memory Management	30
4.1	Overview	30
4.2	Data Structures: The Bitmap	30
4.3	Page Allocation	31
4.3.1	Limitations	32
4.4	Page Deallocation	32
5	Process Management	34
5.1	Supporting Structures and Functions	34
5.1.1	The <code>proc</code> Structure	34
5.1.2	Process Hierarchy and Management Structures	35
5.1.3	Utility Functions and Macros	35
5.1.4	Defined Constants	38
5.2	System Calls Implementation	38
5.2.1	System Call: <code>sys_getpid</code>	39
5.2.2	System Call: <code>sys_waitpid</code>	40
5.2.3	System Call: <code>sys_exit</code>	42
5.2.4	System Call: <code>sys_fork</code>	43
5.2.5	System Call: <code>sys_execv</code>	46
6	Results and Validation	51
6.1	Overview	51
6.2	System Validation	51
6.2.1	Process Management: <code>forktest</code>	51
6.3	Error Handling: The <code>badcall</code> Suite	51
6.3.1	Process Management Tests	52
6.3.2	File System Tests	52
6.4	Shell and Interactive Testing	55

List of Figures

6.1	Output of <code>testbin/forktest</code> showing concurrent parent/child execution.	52
6.2	The <code>badcall</code> test menu.	52
6.3	Validation of process management error handling.	53
6.4	Basic File I/O <code>badcall</code> results.	53
6.5	Advanced File System <code>badcall</code> results.	54
6.6	Execution of the shell and reading a file via <code>cat</code>	55

Chapter 1

Introduction

1.1 Summary

The purpose of this project is to extend the **OS/161** educational operating system to support the concurrent execution of multiple processes from compiled user programs stored on disk. These programs are loaded into OS/161 and executed in user mode under the control of the kernel and the command shell (**bin/sh**).

The implementation focuses primarily on enabling user program execution through the correct functioning of system calls such as **execv** and **dup2**. The project is also limited to the **EMUFS** (emulated file system), which provides a simplified environment for testing file-related operations within OS/161.

1.2 Environment

By default, OS/161 provides only partial support for running executables. To implement multiprogramming and proper user-level execution, it is essential to understand several key components of the kernel and user space. The main kernel files involved in this project are **loadelf.c**, **runprogram.c**, and **uio.c**.

kern/syscall/loadelf.c

This file contains the functions responsible for loading an **ELF (Executable and Linkable Format)** binary from the file system into a process's virtual address space. ELF is the standard format for executable files produced by **os161-gcc**.

In the minimal implementation of OS/161, virtual memory is not fully supported. Although there is a basic translation between the virtual addresses used by executables and physical memory, no mechanism exists to extend memory beyond what is physically available.

kern/syscall/runprogram.c

This file defines the `runprogram()` function, which is responsible for launching a program directly from the kernel menu. The `execv` system call builds upon and extends this function, adding the logic required to properly initialize a user process, set up the address space, and establish the standard file descriptors for input, output, and error streams.

kern/lib/uio.c

This file provides the mechanisms for transferring data between user space and kernel space through the `uio` structure. Proper handling of data movement across these two memory spaces is critical for the correct implementation of system calls such as `read`, `write`, and `getcwd`.

1.3 Traps and System Calls

Exceptions, or traps, form the fundamental mechanism through which the processor transfers control to the operating system. They provide the entry point for handling hardware interrupts, faults, and software requests such as system calls.

When OS/161 boots, it installs an **exception handler** — a piece of assembly code located at a fixed memory address. When a trap occurs, control is transferred to this handler, which saves the current state into a trapframe and passes execution to the kernel's C-level handler for further processing.

In OS/161 terminology, both interrupts and system calls are considered *traps*. System calls, in particular, are triggered when a user program executes a special instruction to request a service from the kernel. These calls are managed by the `syscall/syscall.c` file, which dispatches each call to the appropriate kernel implementation.

locore/trap.c

This file contains low-level trap handling routines. The `mips_trap()` function is responsible for transferring control from user mode to the kernel, while `enter_new_process()` performs the reverse operation when returning to user space. The function `kill_curthread()` is used to terminate faulty user processes that trigger unrecoverable exceptions.

syscall/syscall.c

The central entry point for system call handling. The `syscall()` function interprets system call numbers from the trapframe and delegates execution to the corresponding kernel routine. All system calls developed for this project—covering file management, process control, and exception handling—are implemented or referenced here.

user/lib/crt0/mips

This directory contains the user program startup code. The file `crt0.S` holds MIPS assembly instructions that execute first when a user-level program starts. It initializes the runtime environment and calls the program's `main()` function.

user/lib/libc

This directory contains the user-level C library (`libc`), which provides high-level wrappers around system calls, allowing user programs to interact with the kernel using familiar C functions.

user/lib/libc/unix/errno.c

Defines the global variable `errno`, used by user programs to identify errors returned by system calls.

user/lib/libc/arch/mips/syscalls-mips.S

Contains machine-dependent assembly code for invoking system calls on MIPS.

The `syscalls.S` file, generated at compile time from this source via the `gensyscalls.sh` script, consolidates all user-level system call stubs into a single file, simplifying the build process.

1.4 Problem Definition

The objective of this project is to implement the system calls required for file management, process control, and exception handling. The complete list of system calls to be implemented is defined in `kern/include/kern/syscall.h`:

- **File-related system calls:** `open`, `read`, `write`, `lseek`, `close`, `dup2`, `chdir`, `getcwd`
- **Process and management calls:** `getpid`, `fork`, `execv`, `waitpid`, `_exit`

Each system call must handle errors as specified in the OS/161 manual pages, returning appropriate error codes to maintain compatibility with the grading scripts used for automated testing.

At user level, system call interfaces are declared in `user/include/unistd.h`, while their kernel-side prototypes are found in `kern/include/syscall.h`. The numeric identifiers associated with each call are defined in `kern/include/kern/syscall.h`.

1.5 System Call Concurrency and File Descriptors

A key design consideration is whether two distinct user processes can execute system calls concurrently. In OS/161, the kernel ensures that such operations are correctly synchronized using locks and reference counting mechanisms, preventing race conditions or data corruption.

File descriptors 0, 1, and 2 correspond to **standard input (stdin)**, **standard output (stdout)**, and **standard error (stderr)** respectively. By default, these are attached to the console device, but through the use of `dup2`, processes can redirect them to different files or devices.

A major aspect of this implementation involves designing and managing the data structures responsible for tracking open files and their descriptors across processes.

1.6 Process Management System Calls

`getpid`

Each process in OS/161 is assigned a unique Process ID (PID). The implementation of `getpid` ensures that the kernel can allocate and reclaim PIDs efficiently, preventing exhaustion of the PID space during long executions.

`fork`, `execv`, `waitpid`, `__exit`

These system calls form the foundation of **multiprogramming** in OS/161.

- `fork()` creates a new process by duplicating the parent process, assigning a new PID, and returning distinct values to the parent (child PID) and the child (0).
- `execv()` replaces the process's current address space with a new program loaded from disk, allowing execution of a different binary.

- `waitpid()` allows a parent process to wait for the termination of a specific child process, retrieving its exit status.
- `_exit()` terminates the calling process, releasing its resources and notifying the parent process if it is waiting.

kill__curthread

Handles the termination of a process that encounters a fatal exception or unrecoverable user-mode error. Since the user-space state cannot be trusted after such an exception, this function ensures a safe cleanup of resources.

1.7 Testing

The primary testing environment is the user-level shell located at `user/bin/sh`. When executed, the shell displays a prompt and allows the user to issue simple commands to test system call implementations. By running programs from the shell, the correctness and stability of the file and process management system calls can be verified in a realistic multi-process environment.

1.8 Project Structure

- **Chapter 2 - Kernel Configuration:** This section is dedicated to the configuration of the kernel to be able to use the implemented system calls, supported to newly implemented structures.
- **Chapter 3 - File Handling:** Each of the system call for file handling is detailed in its implementation and also provides an overview of the structures and functions used to support the development of the system calls.
- **Chapter 4 - Virtual Memory Management:** This chapter details the enhancement of the default memory manager (`dumbvm`) through the implementation of a bitmap-based allocator. This mechanism enables the tracking and reclamation of physical pages, effectively preventing memory exhaustion during the execution of multiple processes.
- **Chapter 5 - Process Management:** This chapter presents the design and implementation of the system calls for process management. Additionally, it describes the data structures, synchronization mechanisms, and utility functions that support process creation, execution, termination, and parent-child synchronization.

- **Chapter 6 - Results and Validation:** This chapter presents the validation of the system using standard OS/161 test suites, including `badcall`, `forktest`, and `bigseek`. It analyzes the execution results to confirm the correctness of argument handling, the robustness of error codes, and the stability of the kernel under concurrent workloads.

Chapter 2

Kernel Configuration

2.1 Configuration File

In order to make the requirements of this code available, a few lines were added to the file *conf.kern*, which can be found in the *kern\conf* folder, that not only include the new files *c2_syscalls* and *c2_syscalls_proc* but also define the new option *OPT_C2OS*, which is going to be used to make sure that the include of the file was successful and that they can be used in the code.

```
#####  
#                                     #  
#             C2 Project             #  
#                                     #  
#####  
defoption c2os  
  
optfile c2os syscall/c2_syscalls.c  
optfile c2os syscall/c2_syscalls_proc.c  
optfile c2os syscall/exec.c
```

Listing 2.1: C2 Project Options

This condition, if correctly used, allows the usage of the software without interruption if those files were not included as intended. In the file *kern\compile\C2PROJECT\opt-c2os.h* the developer can check the status of the flag *OPT_C2OS*, which is going to be set to 1 in case of success and 0 otherwise, as illustrated in Listing 2.2. Once this flag is defined, it can be used in the code to define a region that is going to be run only if the value is set to one, as illustrated in Listing 2.3 where the option is used to define the signatures of the system calls that are not available in the default implementation of the *os161* operating system.

```
/* Automatically generated; do not edit */
#ifndef _OPT_C2OS_H_
#define _OPT_C2OS_H_
#define OPT_C2OS 1
#endif /* _OPT_C2OS_H_ */
```

Listing 2.2: C2OS Flag

```
#if OPT_C2OS
int sys_open(userptr_t pathName, int openFlags, mode_t modeFile,
             int32_t *returnVal);
...
#endif
```

Listing 2.3: C2OS Flag Usage Overview

2.2 System Call Dispatcher

The system call dispatcher is responsible for transferring control from user mode into the kernel whenever a process executes the *syscall* instruction. In OS/161 this logic is implemented in the *syscall* function, which receives a pointer to a *trapframe*, a snapshot of the user-level CPU state at the moment the exception occurred.

2.2.1 Role of the Trapframe

A trapframe contains the complete CPU register state at the time the user program invoked the system call. This includes the program counter, stack pointer, the argument registers *a0-a3*, and the return value registers *v0-v1*. The dispatcher uses this structure both to read system call arguments from user mode and to write back the final result that will be returned to the caller.

Some key uses of the *trapframe* in the dispatcher are:

- **Identifying the system call:** The system call number is stored in register *v0*. The dispatcher reads *tf->tf_v0* to determine which kernel routine to call.
- **Retrieving arguments:** The first four 32-bit arguments are passed through registers *a0-a3*, which the dispatcher accesses directly from the trapframe. For 64-bit arguments (e.g., the offset in *lseek*), the dispatcher reconstructs the 64-bit value by combining registers or by fetching additional words from the user stack following the MIPS calling convention.

- **Returning values:** The dispatcher writes the system call's return value into *v0* and *v1* when needed. It also sets *a3* to indicate success *0* or failure *1*. User-level *libc* uses this convention to set *errno* appropriately.
- **Advancing the program counter:** Before returning to user space, the dispatcher increments *tf->tf_epc* by 4. This ensures the CPU resumes at the instruction following the *syscall*; otherwise the system call would repeat indefinitely.

2.2.2 Dispatching System Calls

The dispatcher uses a large *switch* statement to map a system call number to its kernel implementation. For example:

```
case SYS_open:
    err = sys_open(
        (userptr_t)tf->tf_a0,
        tf->tf_a1,
        tf->tf_a2,
        &retval);
    break;
```

Listing 2.4: System Call Open Example

In the base OS/161 distribution only a few system calls are implemented. For this project, however, additional system calls required by the shell were added under the *OPT_C2OS* configuration flag. These include:

- File I/O: *read*, *write*, *open*, *close*, *remove*
- Directory operations: *chdir*, *__getcwd*
- File descriptor manipulation: *lseek*, *dup2*
- Process-related: *getpid*, *waitpid*, *__exit*, *fork*, *execv*

Each case extracts arguments from the trapframe, invokes the corresponding *sys_XXX_SHELL* implementation, and stores the resulting return value back into the trapframe.

2.2.3 Error Handling and Return Conventions

Each system call implementation returns either:

- 0 (success), or
- a positive *errno* value (failure).

The dispatcher sets:

- *tf->tf_v0* to the return value or error code,
- *tf->tf_a3* to 0 on success or 1 on failure.

This convention matches the user-level syscall stubs in *syscalls-mips.S*, allowing them to return -1 and set *errno* when an error occurs.

2.2.4 Entering a Forked Child Process

The dispatcher also supports the *fork* system call function. After creating a new process, the kernel provides the child with a copied trapframe so that it resumes execution as if returning from *fork*. In the child:

- *v0* is set to 0 to indicate “child process”,
- *a3* is set to 0 (success),
- *epc* is incremented to avoid re-executing the syscall.

Finally, the kernel transitions the child process into user mode using *mips_usermode*.

2.2.5 Concurrency of System Calls

A fundamental design consideration in the implementation of system calls is whether multiple user-level processes can execute system calls concurrently. In this system, the answer is **yes**: two or more processes may be inside the kernel executing system calls at the same time.

A system call represents a controlled transition from user mode to kernel mode via a processor exception, but this transition does not imply exclusive access to the

kernel. Each process has its own *trapframe*, kernel stack, and thread context; therefore, when different processes invoke the *syscall* instruction, the kernel maintains separate execution contexts for each of them. The scheduler may context-switch between threads at any point, including while they are executing kernel code, and blocking operations (such as disk I/O or waiting for a child process) allow other threads and processes to enter and execute system calls concurrently.

For this reason, the kernel must be designed under the assumption that system calls are *reentrant* and may execute in parallel. Shared kernel data structures must therefore be protected using synchronization mechanisms. In this project, this requirement is addressed through the use of fine-grained locks. For example, the process file descriptor table is protected by a process lock, and each open file object contains its own lock to protect shared state such as the file offset and reference count. This ensures correct behaviour when multiple processes or threads concurrently perform operations such as *read*, *write*, *close*, or *dup2* on shared file objects.

This design avoids global serialization of system calls, which would unnecessarily limit performance and scalability, and instead allows safe parallel execution while preserving correctness through explicit synchronization.

2.2.6 Atomicity of Read and Write Operations

The implementations of the *read* and *write* system calls were designed to operate in a loop, processing the requested transfer in fixed-size chunks rather than in a single operation. This approach avoids allocating excessively large kernel buffers and allows the kernel to safely handle arbitrarily large user requests. However, this design introduces an important correctness concern related to concurrent access to shared file state.

Although a read operation does not modify file contents, it does update the file offset associated with the open file object. Since open file objects may be shared across multiple file descriptors or processes (e.g., via *fork* or *dup2*), concurrent reads or writes without synchronization could interleave and cause the file offset to advance incorrectly. This could result in a process reading from or writing to an unintended position in the file.

To prevent this issue, both *read* and *write* acquire a per-file lock for the entire duration of the operation, including all iterations of the chunk-processing loop. This ensures that the sequence of offset updates is atomic with respect to other system

calls and that each operation observes a consistent view of the file position. As a result, the kernel guarantees correct behavior even in the presence of concurrent system calls, at the cost of limited serialization on a per-file basis rather than a global lock.

Chapter 3

File Handling

3.1 Supporting Structures and Functions

The implementation of the system calls described in this chapter relies on several key kernel data structures and helper functions that provide the necessary abstraction and synchronization mechanisms between user space and the Virtual File System.

3.1.1 The `openfile` Structure

The `openfile` structure defines a single open file entry shared between processes or multiple file descriptors. It keeps track of the `vnode` representing the file, its current position, access mode, and synchronization primitives to ensure thread-safe access. It is defined as follows:

```
struct openfile {
    struct vnode *vn;           // Pointer to the vnode representing
    the file
    off_t offset;              // Current read/write position in
    the file
    unsigned int countRef;      // Reference count for shared open
    instances
    mode_t modeFile;           // File access mode (O_RDONLY,
    O_WRONLY, O_RDWR)
    int openFlags;              // Flags used during open (O_CREAT,
    O_APPEND, etc.)
    struct lock *lockFile;      // Synchronization lock for
    concurrent access
};
```

Each process contains a `fileTable`, which stores pointers to `openfile` structures. At the system level, a `systemFileTable` array is also maintained to track all open files within the operating system. This allows enforcing limits such as `OPEN_MAX` per process and `SYSTEM_OPEN_MAX` globally.

3.1.2 System and Process File Tables

- **`systemFileTable`** — a global array that tracks all open files across the system. Its maximum capacity is defined by `SYSTEM_OPEN_MAX`, ensuring that the kernel does not exceed system-wide file descriptor limits.
- **`curproc->fileTable`** — a per-process table storing pointers to open file descriptors. Each entry refers to an element of the global `systemFileTable`.

Together, these structures enable consistent reference counting and concurrent file access across multiple processes.

3.1.3 Utility Functions and Macros

Several kernel-level functions and macros support the system call implementations:

- **`uio_uinit()`** and **`uio_kinit()`** — initialize `uio` and `iovec` structures for user-space and kernel-space I/O operations respectively. These functions are fundamental to transferring data between user and kernel buffers in calls such as `sys_read`, `sys_write`, and `sys_getcwd`.
- **`copyinstr()`**, **`copyin()`**, and **`copyout()`** — safely copy data between user and kernel address spaces, validating pointers and preventing segmentation faults.
- **`VOP_READ()`**, **`VOP_WRITE()`**, **`VOP_STAT()`**, and **`VOP_ISSEEKABLE()`** — vnode operations that abstract filesystem-level interactions, allowing system calls to perform reads, writes, metadata queries, and seek checks in a filesystem-agnostic manner.
- **`vfs_open()`**, **`vfs_close()`**, **`vfs_lookup()`**, and **`vfs_getcwd()`** — high-level Virtual File System (VFS) functions responsible for opening, closing, resolving, and retrieving filesystem paths.
- **`lock_create()`** and **`lock_acquire()/release()`** — kernel synchronization primitives ensuring mutual exclusion during concurrent file operations such as reads and writes.

3.1.4 Defined Constants

The system also defines key constants that establish operating limits and buffer management policies:

- **SYSTEM_OPEN_MAX** — maximum number of open files system-wide (typically set to $10 \times \text{OPEN_MAX}$).
- **CHUNK_SIZE** — defines the size of kernel buffers for block I/O operations (4 KiB by default).

These constants ensure efficient memory management and prevent excessive resource usage during large read or write operations.

These elements interact closely within the kernel's file management subsystem. The 'openfile' structure connects user-level file descriptors to the underlying 'vnode' objects managed by the Virtual File System (VFS), while helper functions such as 'copyin/out', 'uio' initializers, and VOP operations provide the mechanisms for transferring data safely and abstracting filesystem-specific details. Together, they allow the system calls to remain filesystem-independent while ensuring correctness and consistency during I/O operations.

Together, these structures and functions provide the foundation for implementing reliable and concurrent file system operations within the kernel.

3.2 System Calls Implementation

System calls act as the fundamental interface between user programs and the kernel, allowing user processes to request services such as file manipulation, process control, and memory management. In this section, we focus on system calls implemented within the kernel related to **file handling**:

1. **sys_open**
2. **sys_read**
3. **sys_write**
4. **sys_close**
5. **sys_lseek**
6. **sys_dup2**

7. `sys_chdir`8. `sys_getcwd`

File system calls primarily interface with the Virtual File System (VFS) layer to perform I/O operations.

3.2.1 System Call: `sys_open`

The `sys_open` system call opens a file specified by *pathName* and returns a file descriptor referring to the opened object. The behavior of the call is controlled by *openFlags*, while *modeFile* specifies the permission bits used only when creating a new file.

Function Prototype

```
int sys_open(userptr_t pathName,
             int openFlags,
             mode_t modeFile,
             int32_t *returnVal);
```

Parameters

- **pathName**: user-space pointer to the file pathname.
- **openFlags**: access mode (`O_RDONLY`, `O_WRONLY`, `O_RDWR`) and optional flags such as `O_CREAT`, `O_EXCL`, `O_TRUNC`, and `O_APPEND`.
- **modeFile**: permission bits for newly created files (used only with `O_CREAT`).
- **returnVal**: pointer where the new file descriptor is stored.

Implementation Summary

The system call first validates the *pathName* pointer and copies the pathname into a kernel buffer using `copyinstr`. The file is then opened via `vfs_open`, which handles all creation and truncation semantics. After a successful open, an `openfile` structure is allocated and initialized, storing the vnode, current offset, reference count, access mode, and a per-file lock.

```

/* 1. Allocate kernel buffer */
kernB = (char *) kcalloc(PATH_MAX);
if (kernB == NULL) return ENOMEM;

/* 2. Copy path from user space */
err = copyinstr((const_userptr_t) pathName, kernB, PATH_MAX, &len)
;
if (err) {
    kfree(kernB);
    return err;
}

```

Listing 3.1: Kernel buffer allocation and copy from user space

The access mode is derived by masking *openFlags* with `O_ACCMODE`. If `O_APPEND` is set, the file offset is initialized to the end of the file using `VOP_STAT`. The open file is then inserted atomically into the current process file table, starting from descriptor 3. If the table is full, `EMFILE` is returned.

On success, the allocated file descriptor is stored in *returnVal*, and the system call returns 0.

```

lock_acquire(curproc->p_lock);
for (int i = 3; i < OPEN_MAX; i++) {
    if (curproc->fileTable[i] == NULL) {
        curproc->fileTable[i] = openF;
        fd = i;
        break;
    }
}
lock_release(curproc->p_lock);

```

Listing 3.2: Finding available space in the current process file table

Error Handling

Condition	Error
Invalid user pointer	EFAULT
Memory allocation failure	ENOMEM
Invalid access mode	EINVAL
Process file table full	EMFILE
VFS failure	Propagated error

3.2.2 Closing Files: `sys_close`

The `sys_close` system call closes the file descriptor specified by *fd*. Closing a file descriptor only affects the calling process; other file descriptors referring to the same open file (e.g., after `fork` or `dup2`) are not affected.

Function Prototype

```
int sys_close(int fd);
```

Parameters

- `fd`: file descriptor to be closed.

Implementation Summary

The system call begins by validating the file descriptor. If *fd* is negative, exceeds `OPEN_MAX`, or does not correspond to an open file in the process file table, the function returns `EBADF`.

To avoid race conditions, the process file table is protected by a process-level lock. The entry corresponding to *fd* is atomically detached from the table, ensuring that the process can no longer access the file using that descriptor.

After detaching the descriptor, the file's internal lock is acquired to safely update shared fields. The reference count of the associated `openfile` structure is decremented. If the reference count remains greater than zero, the file is still in use by other processes or file descriptors, and the system call returns successfully.

If the reference count reaches zero, the calling process owns the file object exclusively. The associated `vnode` is closed using `vfs_close`, which decrements the `vnode`'s reference count and may trigger filesystem-level cleanup. Finally, the per-file lock is destroyed, the `openfile` structure is freed, and the system call returns 0.

```
lock_acquire(f->lockFile);
KASSERT(f->countRef > 0);
f->countRef--;
if (f->countRef > 0) {
```

```

    /* Still in use by other processes */
    lock_release(f->lockFile);
    return 0;
}
/* If RefCount == 0, we own the object completely. */
/* Clean up VFS and Memory */
if (f->vn != NULL) {
    vfs_close(f->vn);
}

```

Listing 3.3: Checking if the file is still being used by another process

Error Handling

Condition	Error
Invalid or unopened file descriptor	EBADF

3.2.3 Reading from Files: `sys_read`

The `sys_read` system call reads up to *bufLen* bytes from the file associated with the file descriptor *fd*, starting from the current file offset, and stores the data in the user-space buffer pointed to by *buffer*. The file must be opened with read permissions.

Function Prototype

```

int sys_read(int fd,
             userptr_t buffer,
             size_t bufLen,
             ssize_t *returnVal);

```

Parameters

- **fd**: file descriptor to read from.
- **buffer**: user-space destination buffer.
- **bufLen**: maximum number of bytes to read.
- **returnVal**: pointer where the number of bytes read is stored.

Implementation Summary

The system call first validates the file descriptor and the user buffer pointer. If *fd* is invalid, does not refer to an open file, or if the file was opened in write-only mode, the function returns **EBADF**. A zero-length read immediately returns success with zero bytes read. A temporary kernel buffer of fixed size is allocated to safely transfer data between the file system and user space in chunks. The file's internal lock is acquired to serialize access and protect the shared file offset. For each iteration, a **uio** and **iovec** structure are initialized using **uio_kinit**, and the read operation is performed by calling **VOP_READ**.

```
lock_acquire(fl->lockFile);
while (nRead < buflen)
{
    size_t remaining = buflen - nRead;
    size_t nLen = (remaining < CHUNK_SIZE) ? remaining :
CHUNK_SIZE;
    /* Prepare UIO for Kernel Buffer */
    uio_kinit(&iiov, &ku, kBuffer, nLen, fl->offset, UIO_READ);
    res = VOP_READ(fl->vn, &ku);
    if (res) {
        lock_release(fl->lockFile);
        kfree(kBuffer);
        return res;
    }
    /* Calculate bytes actually read from disk */
    size_t readThisChunk = nLen - ku.uio_resid;
    /* Update Offset (Protected by fl->lockFile) */
    fl->offset += readThisChunk;
    /* If we hit EOF (read 0 bytes), stop */
    if (readThisChunk == 0) break;
    /* Copy data from Kernel Buffer to User Buffer */
    res = copyout(kBuffer, (userptr_t)((uintptr_t)buffer + nRead),
readThisChunk);
    if (res) {
        lock_release(fl->lockFile);
        kfree(kBuffer);
        return res; /* likely EFAULT */
    }
    nRead += readThisChunk;
    /* If we read less than requested (EOF or short read), stop */
    if (readThisChunk < nLen) break;
}
lock_release(fl->lockFile);
```

Listing 3.4: Loop cycle to read

After each successful read, the file offset is updated, and the data is copied from the kernel buffer to the user buffer using `copyout`. The loop terminates when the requested number of bytes has been read, an end-of-file condition is reached, or a short read occurs. On success, the total number of bytes read is stored in *returnVal*, and the system call returns 0.

Error Handling

Condition	Error
Invalid or unopened file descriptor	EBADF
Invalid user buffer	EFAULT
Memory allocation failure	ENOMEM
Filesystem read failure	Propagated error

3.2.4 Writing into Files: `sys_write`

The `sys_write` system call writes up to *bufLen* bytes from the user-space buffer pointed to by *buffer* into the file associated with file descriptor *fd*, starting at the current file offset. The file must be opened with write permissions. Upon success, the file offset is advanced by the number of bytes written.

Function Prototype

```
int sys_write(int fd,
              userptr_t buffer,
              size_t bufLen,
              ssize_t *returnVal);
```

Parameters

- **fd**: file descriptor to write to.
- **buffer**: user-space source buffer.
- **bufLen**: number of bytes to write.
- **returnVal**: pointer where the number of bytes written is stored.

Implementation Summary

The system call first validates its arguments. If *fd* is invalid, does not refer to an open file, or the file was opened in read-only mode, the function returns **EBADF**. A null user buffer results in **EFAULT**. If *bufLen* is zero, the call succeeds immediately with zero bytes written.

To safely transfer data from user space to the filesystem, a fixed-size kernel buffer is allocated, and the write operation is performed in chunks. For each iteration, a portion of the user buffer is copied into kernel space using **copyin**. The file's internal lock is then acquired to serialize access and protect the shared file offset.

If the file was opened with the **O_APPEND** flag, the current end-of-file position is retrieved using **VOP_STAT**, and the file offset is updated accordingly before writing. The kernel prepares a **uio** and **iovec** structure using **uio_kinit**, and the data is written by invoking **VOP_WRITE**. After each successful write, the file offset is advanced by the number of bytes written.

The loop terminates when all requested bytes have been written or when a partial write occurs. Finally, the kernel buffer is freed, the total number of bytes written is stored in *returnVal*, and the system call returns 0.

```
lock_acquire(fl->lockFile);
while (nWrite < bufLen)
{
    size_t remaining = bufLen - nWrite;
    size_t nLen = (remaining < CHUNK_SIZE) ? remaining :
    CHUNK_SIZE;
    res = copyin((const_userptr_t)((uintptr_t)buffer + nWrite),
    kBuffer, nLen);
    if (res)
    {
        kfree(kBuffer);
        lock_release(fl->lockFile);
        return res;
    }
    uio_kinit(&iiov, &ku, kBuffer, nLen, fl->offset, UIO_WRITE);
    if (fl->openFlags & O_APPEND)
    {
        struct stat st;
        res = VOP_STAT(fl->vn, &st);
        if (res)
        {
            kfree(kBuffer);
            lock_release(fl->lockFile);
        }
    }
}
```

```

        return res;
    }
    fl->offset = st.st_size;
    /* Re-init uio with new offset */
    uio_kinit(&iiov, &ku, kBuffer, nLen, fl->offset, UIO_WRITE)
;
}
res = VOP_WRITE(fl->vn, &ku);
if (res)
{
    kfree(kBuffer);
    lock_release(fl->lockFile);
    return res;
}
size_t wrote = nLen - ku.uio_resid;
nWrite += wrote;
fl->offset += wrote;
if (wrote == 0 || wrote < nLen) break;
}
lock_release(fl->lockFile);

```

Listing 3.5: Loop cycle to write

Error Handling

Condition	Error
Invalid or unopened file descriptor	EBADF
File opened read-only	EBADF
Invalid user buffer	EFAULT
Memory allocation failure	ENOMEM
Filesystem write failure	Propagated error

3.2.5 Manipulating File Offsets: `sys_lseek`

The `sys_lseek` system call repositions the current file offset associated with the open file descriptor *fd*. Subsequent read or write operations will occur starting from the new offset. The new position is computed as a 64-bit value based on *pos* and the reference point *whence*.

Function Prototype

```

int sys_lseek(int fd,
              off_t pos,

```

```

int whence,
int32_t *retval_low32,
int32_t *retval_upp32);

```

Parameters

- **fd**: File descriptor whose offset is to be modified.
- **pos**: The 64-bit signed offset value used in the computation.
- **whence**: Reference point (SEEK_SET, SEEK_CUR, SEEK_END).
- **retval_low32**: Pointer to an integer where the lower 32 bits of the result are stored.
- **retval_upp32**: Pointer to an integer where the upper 32 bits of the result are stored.

Implementation Summary

The system call first validates the file descriptor. If *fd* is invalid, NULL, or refers to a non-seekable object (like a console), the appropriate error code (EBADF or ESPIPE) is returned.

Unlike previous iterations, the file lock is acquired **before** determining the new offset. This ensures that any calculation relative to the current offset (as in SEEK_CUR) is atomic and consistent.

The function switches on *whence* to calculate `retvalJoined`. If the calculation results in an invalid position (e.g., negative offset), the lock is released immediately, and EINVAL is returned. For SEEK_END, the file size is retrieved via VOP_STAT.

```

lock_acquire(fl->lockFile); /* Lock before calculation */

switch(whence)
{
    case SEEK_SET:
        if (pos < 0) {
            lock_release(fl->lockFile);
            return EINVAL;
        }
        retvalJoined = pos;
        break;
    case SEEK_CUR:
        if (pos < 0 && -pos > fl->offset) { /* Check underflow */
            lock_release(fl->lockFile);

```

```

        return EINVAL;
    }
    retvalJoined = fl->offset + pos;
    break;
case SEEK_END:
    res = VOP_STAT(fl->vn, &st);
    if(res) {
        lock_release(fl->lockFile);
        return res;
    }
    retvalJoined = st.st_size + pos;
    break;
default:
    lock_release(fl->lockFile);
    return EINVAL;
}

```

Listing 3.6: Whence handling with Locking

If the calculated `retvalJoined` is valid, the file object's offset is updated, and the lock is released.

Finally, to support the 32-bit MIPS architecture, the resulting 64-bit offset is split into two 32-bit values. The upper 32 bits are stored in `retval_upp32` (passed to register `v0`) and the lower 32 bits in `retval_low32` (passed to register `v1`).

Error Handling

Condition	Error
Invalid or unopened file descriptor	EBADF
Non-seekable file (e.g., pipe/console)	ESPIPE
Invalid whence value	EINVAL
Negative resulting offset	EINVAL
Filesystem error (during VOP_STAT)	Propagated error

3.2.6 Duplicating File Descriptors: `sys_dup2`

The `sys_dup2` system call duplicates an existing file descriptor *oldFd* to a new file descriptor *newFd*. The new descriptor refers to the same underlying open file, sharing the file offset and flags. If *newFd* is already open, it is closed before duplication.

Function Prototype

```
int sys_dup2(int oldFd, int newFd, int *returnVal);
```

Parameters

- **oldFd**: file descriptor to duplicate.
- **newFd**: desired file descriptor number for the duplicate.
- **returnVal**: pointer where the duplicated descriptor is stored.

Implementation Summary

The system call first validates both descriptors. If either *oldFd* or *newFd* is invalid, **EBADF** is returned. If the two descriptors are equal, the call succeeds immediately with no changes.

The process file table is locked to ensure thread-safe access. If *oldFd* does not refer to an open file, the function returns **EBADF**. If *newFd* is already open, the existing file is detached from the table and marked for closing after the lock is released, avoiding deadlocks.

The reference count of the old file is incremented, and it is assigned to *newFd* in the process file table. The lock is then released. Any displaced file previously associated with *newFd* is closed using the same logic as **sys_close**, including decrementing its reference count, closing the vnode if needed, destroying its lock, and freeing its memory.

```
/* Cleanup displaced file if it existed */
if (new_of_to_close != NULL) {
    /* Use the same logic as sys_close here */
    bool is_last = false;

    lock_acquire(new_of_to_close->lockFile);
    new_of_to_close->countRef--;
    if (new_of_to_close->countRef == 0) is_last = true;
    lock_release(new_of_to_close->lockFile);

    if (is_last) {
        vfs_close(new_of_to_close->vn);
        lock_destroy(new_of_to_close->lockFile);
        kfree(new_of_to_close);
    }
}
```

```
}  

```

Listing 3.7: Whence handling

Finally, *newFd* is stored in *returnVal*, and the function returns 0 to indicate success.

Error Handling

Condition	Error
Invalid file descriptor(s)	EBADF

3.2.7 Change Current Directory: `sys_chdir`

The `sys_chdir` system call sets the current working directory of the calling process to the directory specified by *pathName*. The kernel ensures the path refers to a valid directory and updates the process's working directory atomically with respect to the assignment, though it cannot guarantee atomicity with respect to other threads accessing *pathName*.

Function Prototype

```
int sys_chdir(userptr_t pathName);
```

Parameters

- **pathName:** user-space pointer to the new directory path.

Implementation Summary

The function first validates *pathName*, returning `EFAULT` if it is `NULL`. A kernel buffer is then allocated to copy the path from user space; failure to allocate results in `ENOMEM`.

Using `vfs_lookup`, the kernel obtains the `vnode` for the given path. If the lookup fails, the buffer is freed and the error returned. The function then calls `VOP_STAT` to check that the `vnode` represents a directory (`S_IFDIR`). If not, `ENOTDIR`

is returned.

On success, the process's current working directory (*curproc->p_cwd*) is replaced with the new vnode. The old vnode is closed using `vfs_close` if it exists. The system call returns 0 to indicate success.

```
err = vfs_lookup(kernB, &vn);
kfree(kernB);
if (err) return err;
struct stat fileStat;
err = VOP_STAT(vn, &fileStat);
if (err)
{
    vfs_close(vn);
    return err;
}
if ((fileStat.st_mode & S_IFDIR) == 0)
{
    vfs_close(vn);
    return ENOTDIR;
}
```

Listing 3.8: VFS lookup and directory validation

Error Handling

Condition	Error
<i>pathName</i> is NULL	EFAULT
Kernel memory allocation fails	ENOMEM
Directory does not exist	Propagated error from <code>vfs_lookup</code>
Path is not a directory	ENOTDIR

3.2.8 Name of Current Working Directory: `sys_getcwd`

The `sys_getcwd` system call retrieves the absolute path of the current working directory and stores it in a user-provided buffer. The number of bytes written is returned in *returnVal*.

Function Prototype

```
int sys_getcwd(userptr_t buffer, size_t bufLen, int *returnVal);
```

Parameters

- **buffer**: user-space buffer to store the current directory path (null-terminated).
- **bufLen**: size of the buffer.
- **returnVal**: pointer to store the number of bytes written.

Implementation Summary

The system call first validates the user pointer *buffer*, returning **EFAULT** if it is **NULL**. A kernel buffer of fixed size is allocated to temporarily hold the pathname, with failure returning **ENOMEM**.

The kernel initializes a **uio** structure via **uio_kinit**, describing the destination in user space. The VFS function **vfs_getcwd** traverses the current working directory (*curproc->p_cwd*) and writes the resulting path into the kernel buffer.

```
result = vfs_getcwd(&ku);  
size_t actualLen = (bufLen < CHUNK_SIZE ? bufLen : CHUNK_SIZE) -  
    ku.uio_resid;  
result = copyout(kBuffer, buffer, actualLen);  
*returnVal = actualLen;
```

Listing 3.9: Read from VFS, copy to user space, and return length

After a successful VFS call, the path is copied from the kernel buffer to the user buffer using **copyout**, and the number of bytes written is stored in *returnVal*. All allocated kernel memory is freed, and the system call returns 0 on success.

Error Handling

Condition	Error
<i>buffer</i> is NULL	EFAULT
Kernel memory allocation fails	ENOMEM
Directory no longer exists	ENOENT
I/O error during traversal	EIO
Failed copy to user memory	EFAULT

Chapter 4

Virtual Memory Management

4.1 Overview

The default virtual memory manager in OS/161, located in `kern/arch/mips/vm/dumbvm.c`, is a minimal implementation known as `dumbvm`. By default, it performs contiguous allocation of physical memory using `ram_stealmem()` but lacks any mechanism to free or reuse that memory. As a result, the system eventually runs out of RAM, leading to kernel panics during extended operations.

In this chapter, we present the implementation of a simple memory management system that extends `dumbvm` to support page reclamation. This was achieved using a bitmap-based strategy to track the status of physical pages, enabling the reuse of memory released by processes.

4.2 Data Structures: The Bitmap

To manage the state of the RAM pages, we introduced two auxiliary data structures. These are dynamically allocated during the system bootstrap phase in `vm_bootstrap`.

- `bitmapFreePages`: An array of integers where each index corresponds to a physical page frame.
 - **0**: The page is occupied or currently unmanaged (initially, all pages are set to 0).
 - **1**: The page has been freed and is available for reuse.

- **bitmapSizePages:** An array of `unsigned long` used to store the size of an allocation. If a process requests N pages starting at index i , `bitmapSizePages[i]` stores the value N . This allows the deallocation function to know exactly how many pages to release.

```

1  /* Number of ram pages at bootstrap */
2  static int nRamPages = 0;
3  /* Bitmap storing the position of the free pages */
4  static unsigned int *bitmapFreePages = NULL;
5  /* Bitmap storing the size of the free pages */
6  static unsigned long *bitmapSizePages = NULL;

```

Listing 4.1: Bitmap Data Structures

During initialization, `vm_bootstrap` calculates the total number of RAM pages available and allocates the bitmaps. They are initialized to 0, which implies that initially, the bitmap is empty of "free" pages. The system relies on `ram_stealmem` to acquire fresh memory until pages are freed and added to the pool.

4.3 Page Allocation

The core allocation logic is handled by the `alloc_kpages` routine, which calls `getppages`. The enhanced implementation attempts to satisfy memory requests in two steps:

1. **Reuse (Bitmap Search):** The function `getfreeppages` scans `bitmapFreePages` for a contiguous block of reusable pages (marked as 1).
2. **Fresh Allocation:** If the bitmap does not contain a sufficient contiguous block, the system falls back to the original `ram_stealmem` mechanism.

Regardless of the allocation source, the size of the allocated block is recorded in `bitmapSizePages` to ensure it can be properly freed later.

The Search Algorithm

The `getfreeppages` function implements a linear scan ($O(N)$) to find a contiguous block of free pages. It uses a "sliding window" approach to track the start of a potential sequence.

```

1  for (long int i = 0; i < nRamPages; i++) {
2      /* Check if page is free (1) */
3      if (bitmapFreePages[i]) {
4          /* If start of a new run, record start index */

```

```

5         if (i == 0 || bitmapFreePages[i-1] == 0) {
6             tmpAddr = i;
7         }
8         /* Check if run length is sufficient */
9         if (i - tmpAddr + 1 >= (long int)npages) {
10             startAddr = tmpAddr;
11             break;
12         }
13     }
14 }

```

Listing 4.2: Linear Search for Free Pages

This approach allows finding a suitable block in a single pass. The access to these structures is protected by a spinlock (`freemem_lock`) to ensure thread safety and prevent race conditions during concurrent memory requests.

4.3.1 Limitations

One limitation of this "reuse-only" strategy is fragmentation. Since `ram_stealmem` is strictly linear and the bitmap only captures freed pages, we cannot coalesce a freed page at the very end of the bitmap with the fresh memory remaining in `ram_stealmem`. However, for the scope of this project, this simple recycling significantly extends the system's uptime.

4.4 Page Deallocation

Memory release is handled by `free_kpages`. This function first converts the virtual address to a physical address (by subtracting `MIPS_KSEG0`). It then retrieves the size of the allocation from `bitmapSizePages` using the page index.

The actual deallocation is performed by `freeppages`, which updates the `bitmapFreePages` array. It sets the bits corresponding to the released range to `1`, marking them as available for future allocations.

```

1 static void freeppages(paddr_t paddr, unsigned long size) {
2     /* ... setup ... */
3     unsigned long startAddr = paddr/PAGE_SIZE;
4
5     spinlock_acquire(&freemem_lock);
6
7     /* Mark pages as free (1) */
8     for (unsigned long i = startAddr; i < startAddr + size; i++) {
9         bitmapFreePages[i] = 1;
10    }
11
12    spinlock_release(&freemem_lock);

```

13 | }

Listing 4.3: Releasing Pages

This mechanism effectively transforms `dumbvm` from a strictly linear allocator into a system capable of recycling memory, which is essential for the stability of the kernel during the execution of user processes.

Chapter 5

Process Management

5.1 Supporting Structures and Functions

The implementation of the process management system calls described in this chapter relies on several essential kernel data structures and helper functions that coordinate process creation, execution, synchronization, and termination. These structures and functions provide the abstraction and mechanisms required to manage process lifecycles efficiently within OS/161.

5.1.1 The `proc` Structure

The `proc` structure defines a single process within the OS/161 kernel. It encapsulates all essential information required to manage process execution, address spaces, file descriptors, and parent-child relationships. This structure serves as the backbone of process management and provides synchronization mechanisms to ensure safe concurrent access by multiple kernel threads. It is defined as follows:

```
struct proc {
    char *p_name;           // Process name
    struct spinlock p_lock; // Protects process fields
    unsigned p_numthreads;  // Number of threads in the
    process
    struct addrspace *p_addrspace; // Virtual address space
    struct vnode *p_cwd;          // Current working directory

    // C20S extensions
    int p_status;           // Exit status of the process
    pid_t p_pid;            // Unique process ID
    pid_t parent_pid;       // Parent process ID
}
```

```

    struct child_list *children_list; // Linked list of child
    processes
    struct cv *p_cv;                  // Condition variable for waitpid
    ()
    struct lock *p_locklock;          // Lock protecting process status
    struct openfile *fileTable[OPEN_MAX]; // Per-process open file
    table
};

```

Each process has its own address space, open file table, and synchronization primitives for inter-process communication. The `p_pid` uniquely identifies each process in the system, while `parent_pid` and `children_list` maintain the hierarchical relationship between processes. The inclusion of `p_cv` and `p_locklock` enables proper synchronization between parent and child processes.

5.1.2 Process Hierarchy and Management Structures

- **processTable** — a global process table that keeps track of all active processes in the system. It allows the kernel to locate processes by their PID and ensures efficient PID allocation.
- **curproc** — a pointer to the currently executing process.
- **children_list** — a per-process linked list that records all direct child processes. It enables synchronization between parent and child processes.

Together, these structures enable robust process creation, termination, and synchronization. The combination of the global `procTable` and per-process hierarchical data ensures that the kernel can efficiently manage process lifecycles, enforce parent-child relationships, and support concurrent execution in a multiprogramming environment.

5.1.3 Utility Functions and Macros

Several kernel-level functions and synchronization primitives provide essential support for process management and control operations:

- **proc_init()** — assigns a unique process identifier (PID) to a newly created process and inserts it into the global process table. Ensures atomic updates through spinlock protection.

- **proc_deinit()** — removes a process entry from the process table, releases its synchronization primitives, and cleans up its child relationship and associated resources.
- **proc_create()** — allocates and initializes a new process structure, setting up fields such as the process name, PID, and synchronization primitives.
- **proc_destroy()** — deallocates all resources associated with a process, including its address space, file table, and locks.
- **as_copy()** — creates an exact duplicate of a process's address space, including code, data, and stack segments.
- **copyin()** and **copyout()** — transfer fixed-size data blocks between user space and kernel space safely.
- **copyinstr()** — copies a null-terminated string from user space into a kernel buffer, ensuring proper memory access validation.
- **cv_wait()** — suspends the calling thread on a condition variable until it is signaled.
- **cv_signal()** — wakes a single thread waiting on a condition variable.
- **lock_acquire()** and **lock_release()** — provide mutual exclusion when accessing shared kernel data structures.
- **proc_search()** — retrieves a process structure based on its process identifier (PID).
- **is_child()** — determines whether a given process is a child of another process.
- **destroy_child_list()** — Destroys the child list of a parent process which is being destroyed.
- **add_new_child()** — appends a new child process entry to a parent's child list.
- **remove_child_from_list()** — removes a child process entry from a parent's child list once it terminates.
- **thread_fork()** — creates a new kernel thread to execute concurrently within the system.
- **thread_exit()** — cause the current thread to exit.

- **proc_addthread()** and **proc_remthread()** — attaches and detaches a thread to a process structure.
- **call_enter_forked_process()** — prepares and transitions a newly created process into user mode for execution.
- **enter_new_process()** — transfers control from the kernel to user mode, initializing registers, stack, and entry point for a new program.
- **exec_create_sem()** — initializes a counting semaphore used to throttle concurrent process executions, preventing resource exhaustion during large allocations.
- **argbuf_allocate()** — allocates a kernel buffer used to store argument strings and metadata during program loading.
- **argbuf_copyin()** — copies argument strings from user space into a kernel buffer, ensuring safe and sequential extraction of data.
- **argbuf_fromuser()** — constructs a complete argument buffer from user memory, expanding its capacity dynamically if necessary.
- **argbuf_copyout()** — writes argument strings and pointer arrays into the new user stack in preparation for process execution.
- **argbuf_cleanup()** — releases memory allocated for the argument buffer and resets internal counters and flags.
- **loadexec()** — loads an executable binary into a newly created address space, sets up a user stack, and prepares entry and stack pointers for execution. It also updates the current thread's name to match the loaded program.
- **as_create()** and **as_destroy()** — manage the lifecycle of address spaces, providing isolation between user processes.
- **load_elf()** — interprets and loads an ELF-format executable file into memory, initializing the process entry point.
- **proc_setas()**, **proc_getas()** and **as_activate()** — change, fetch and activate the address space of the current process.

5.1.4 Defined Constants

The process management subsystem defines several key constants that determine process handling behavior, system limits, and memory layout policies:

- **PROC_MAX** — defines the maximum number of simultaneously running processes in the system, limiting resource usage and PID allocation range.
- **PAGE_SIZE** — represents the size of a memory page (4 KiB by default), used for address space alignment and buffer allocation within the virtual memory subsystem.
- **WNOHANG** — specifies non-blocking behavior for `waitpid()`, allowing the system call to return immediately if no child process has terminated.
- **WUNTRACED** — enables `waitpid()` to report status changes not only for exited child processes but also for those that have been stopped.
- **WAIT_ANY** — a special value passed to `waitpid()` to wait for any child process to terminate.
- **WAIT_MYPGRP** — indicates that the call should wait for any process belonging to the same process group.

These constants provide control and predictability over process lifecycle management, ensuring proper synchronization, memory allocation, and enforcement of kernel-imposed limits on process creation and tracking. They serve as essential parameters for maintaining stability and consistency across the process management subsystem.

5.2 System Calls Implementation

System calls serve as the primary interface between user programs and the kernel, enabling processes to request kernel-level operations such as creation, execution, synchronization, and termination. In this section, we focus on system calls implemented within the kernel related to **process management**:

1. `sys_getpid`
2. `sys_waitpid`

3. `sys__exit`
4. `sys__fork`
5. `sys__execv`

Process management system calls form the foundation of multiprogramming within the OS/161 kernel. They enable the creation of new processes, control over process hierarchies, execution of user programs, and synchronization between parent and child processes. Together, these calls provide the core mechanisms for process lifecycle management and inter-process coordination.

5.2.1 System Call: `sys_getpid`

The `sys_getpid` system call retrieves the unique process identifier (PID) of the currently executing process. It provides a user-level program with a means to identify itself to the kernel and other processes.

Function Prototype

```
int sys_getpid(pid_t *retvalpid);
```

Parameters

- **retvalpid**: pointer to a variable where the current process PID will be stored.

Implementation Summary

The implementation verifies that a valid process context exists by checking the global `curproc` pointer. If the current process is active, its PID (`p_pid`) is stored in the memory location referenced by `retvalpid`.

The operation does not involve any user-kernel data transfers beyond this simple value assignment, as the process ID is an internal kernel property readily available in the process structure. On successful completion, the function returns zero.

```
int sys_getpid(pid_t *retvalpid) {  
    if (curproc != NULL) {  
        *retvalpid = curproc->p_pid;  
    }  
}
```

```
    return 0;
}
```

Listing 5.1: Retrieving the current process identifier

5.2.2 System Call: `sys_waitpid`

The `sys_waitpid` system call allows a process to wait for a specific child process to terminate. It provides synchronization between a parent and its child processes and enables retrieval of the child's exit status.

Function Prototype

```
int sys_waitpid(pid_t pid,
                int *status,
                int options,
                int32_t *retval);
```

Parameters

- **pid:** the process identifier of the child process to wait for.
- **status:** pointer to an integer where the child process's exit status will be stored.
- **options:** flags that modify the behavior of the wait (e.g., `WNOHANG` for non-blocking wait).
- **retval:** pointer to store the PID of the child process that has been waited on.

Implementation Summary

The system call first validates the `pid` to ensure it refers to a child of the current process. It then locates the child process structure in the process table.

```
/* 1. Filter Invalid PIDs */
if (pid == curproc->p_pid) return ECHILD;

/* 2. Check if Child Exists in our list */
if (is_child(curproc, pid) == -1) return ECHILD;
```

```

/* 3. Handle Options (WNOHANG) */
if (options != 0 && options != WNOHANG) return EINVAL;

/* 4. Find the Process Structure */
struct proc *proc = proc_search(pid);
if (proc == NULL) return ESRCH;

```

Listing 5.2: Validate and locate the child process

If the child process is still running, `sys_waitpid` suspends the calling process by sleeping on the child's condition variable, unless the `WNOHANG` option is specified, in which case it returns immediately with a status of zero.

```

lock_acquire(proc->p_locklock);

if (proc->p_numthreads > 0) {
    /* Process is still running */
    if (options == WNOHANG) {
        lock_release(proc->p_locklock);
        *status = 0;
        *retval = pid;
        return 0;
    }

    /* Wait for it to die */
    while (proc->p_numthreads > 0) {
        cv_wait(proc->p_cv, proc->p_locklock);
    }
}
lock_release(proc->p_locklock);

```

Listing 5.3: Waiting for a child process to terminate

Once the child process has terminated, its exit status is retrieved and copied safely to the user-provided memory location. The child is then removed from the parent's child list, and its process structure is destroyed, completing the reaping process. On success, the PID of the terminated child is returned in `retval`.

```

/* 6. Process is dead. Extract status safely. */
int exit_code = proc->p_status;

/* IF user provided a status pointer, copy it out safely */
if (status != NULL) {
    /* copyout(src, dest, len) */
    result = copyout(&exit_code, (userptr_t)status, sizeof(int));
}

```

```

        if (result) {
            /* If copyout fails (EFAULT), we still successfully
            waited for the process.
            * But we can't return the status. Standard behavior
            is to return EFAULT.
            * CRITICAL: We must arguably still destroy the zombie
            to prevent leaks,
            * or leave it depending on strict POSIX interp.
            * For OS161, returning EFAULT is usually enough.
            */
            return result;
        }
    }

/* 7. Cleanup and Return */
    *retval = pid;

/* Remove from parent's child list */
    remove_child_from_list(curproc, pid);

/* Finally, destroy the process structure (The Reaping) */
    proc_destroy(proc);

```

Listing 5.4: Retrive value and clean up

Error Handling

Condition	Error
Specified PID is not a child	ECHILD
Invalid options value	EINVAL
Child process does not exist	ESRCH
Failed to copy exit status to user memory	EFAULT

5.2.3 System Call: `sys_exit`

The `sys_exit` system call terminates the currently executing process. It provides a mechanism for a process to signal its completion to its parent and release all associated resources. This function does not return to the calling context.

Function Prototype

```
void sys_exit(int exitcode);
```

Parameters

- **exitcode**: integer value representing the termination status of the process. This status is communicated to the parent process and can be retrieved via `sys_waitpid`.

Implementation Summary

The system call first stores the process exit status in the process structure using the `_MKWAIT_EXIT` macro. It then signals the parent process through the condition variable associated with the child, indicating that the process has completed. Finally, `thread_exit` is called to terminate the current thread and clean up the execution context.

```
struct proc *p = curproc;

p->p_status = _MKWAIT_EXIT(exitcode);

lock_acquire(p->p_locklock);
cv_signal(p->p_cv, p->p_locklock);
lock_release(p->p_locklock);

thread_exit();
panic("sys_exit: Should not return");
```

Listing 5.5: Setting exit status and notifying parent

5.2.4 System Call: `sys_fork`

The `sys_fork` system call creates a new process by duplicating the currently executing process. It provides the mechanism for process creation and supports multiprogramming by allowing concurrent execution of parent and child processes.

Function Prototype

```
int sys_fork(struct trapframe *ctf, pid_t *retval);
```

Parameters

- **ctf**: pointer to the trapframe of the current thread, which is copied into the child to preserve the CPU state.
- **retval**: pointer to a variable where the PID of the newly created child process is stored.

Implementation Summary

The system call first allocates a new process structure for the child. It duplicates the parent's address space and copies the trapframe to preserve the CPU context. The current working directory is also replicated, ensuring that the child starts with the same filesystem context.

```
struct proc *parent = curproc;
struct proc *child = NULL;
struct trapframe *child_tf = NULL;
int err;

KASSERT(curproc != NULL);
KASSERT(ctf != NULL);
KASSERT(retval != NULL);

/* 1. Create Child Process */
child = proc_create(parent->p_name);
if (child == NULL) {
    return ENOMEM;
}

err = as_copy(parent->p_addrspace, &child->p_addrspace);
if (err) {
    proc_destroy(child);
    return err;
}

/* 3. Copy Trapframe */
child_tf = kmalloc(sizeof(struct trapframe));
if (child_tf == NULL) {
    proc_destroy(child);
    return ENOMEM;
}
memmove(child_tf, ctf, sizeof(struct trapframe));

/* 4. Copy Current Working Directory (CWD) */
spinlock_acquire(&parent->p_lock);
if (parent->p_cwd != NULL) {
```

```

        VOP_INCREP(parent->p_cwd);
        child->p_cwd = parent->p_cwd;
    }
    spinlock_release(&parent->p_lock);

```

Listing 5.6: Creating a child process and duplicating parent resources

The child shares the parent’s open file table via a shallow copy, with reference counts incremented to maintain proper resource management.

```

for (int i = 0; i < OPEN_MAX; i++) {
    struct openfile *file = parent->fileTable[i];

    if (file != NULL) {
        /* Shallow Copy: Point to the SAME openfile struct */
        lock_acquire(file->lockFile); // Safety while
        modifying refcount

        child->fileTable[i] = file;
        file->countRef++; // Increment reference count

        lock_release(file->lockFile);
    }
}

```

Listing 5.7: Sharing the file table

The child is linked as a child process of the parent and added to the parent’s child list.

```

/* 6. Link Parent */
child->parent_pid = parent->p_pid;

/* 7. ADD TO PARENT’S LIST */
if (add_new_child(parent, child->p_pid) == -1) {
    kfree(child_tf);
    proc_destroy(child);
    return ENOMEM;
}

```

Listing 5.8: Linking child to parent

A new thread is created to execute the child process, using the copied trapframe to start execution. Upon success, the child’s PID is returned via `retval`.

```

err = thread_fork(
    parent->p_name,

```

```
        child,
        call_enter_forked_process,
        (void *) child_tf,
        (unsigned long) 0
    );

    if (err) {
        kfree(child_tf);
        proc_destroy(child);
        return err;
    }

    *retval = child->p_pid;
```

Listing 5.9: Creating a new thread for the child and returning its PID

Error Handling

Condition	Error
Memory allocation failure	ENOMEM

5.2.5 System Call: `sys_execv`

The `sys_execv` system call replaces the currently executing process image with a new program. It provides a mechanism for a process to load and execute a different executable while preserving the same PID. The call sets up the new user stack and arguments, discarding the previous address space.

Function Prototype

```
int sys_execv(const char *progname, char *argv[]);
```

Parameters

- **progname:** path to the executable program to load.
- **argv:** array of arguments to pass to the new program; the array is NULL-terminated.

Implementation Summary

The system call first copies the program name from user space into a kernel buffer. It then allocates and fills a kernel argument buffer from the user-provided argument array.

```
int result, argc;
vaddr_t entrypoint, stackptr;
userptr_t user_prog = (userptr_t) progname;
userptr_t user_argv = (userptr_t) argv;

char *kprog = kmalloc(PATH_MAX);
if (kprog == NULL) return ENOMEM;

result = copyinstr(user_prog, kprog, PATH_MAX, NULL);
if (result) {
    kfree(kprog);
    return result;
}

struct exec_args args = {0};
/* Assuming argbuf_init is not needed or handled by struct
init */

result = argbuf_fromuser(&args, user_argv);
if (result) {
    argbuf_cleanup(&args);
    kfree(kprog);
    return result;
}
```

Listing 5.10: Copying the program name and arguments

The new executable is loaded and the old address space of the process is destroyed. A new user stack is created, and the arguments are copied onto it in preparation for execution. The loading of the executable, the destroying of the process and the creation of the new stack are all done in the `loadexec` function.

```
result = loadexec(kprog, &entrypoint, &stackptr);

kfree(kprog);
if (result) {
    argbuf_cleanup(&args);
    return result;
}

result = argbuf_copyout(&args, &stackptr, &argc, &user_argv);
```

```

    if (result) {
        panic("sys_execv: argbuf_copyout failed: %s\n", strerror(
result));
    }

    argbuf_cleanup(&args);

```

Listing 5.11: Calling loadexec and copying the arguments onto the stack

```

    struct addrspace *new_as = NULL;
    struct addrspace *prev_as = NULL;
    struct vnode *vnode = NULL;
    char *proc_name = NULL;
    int result;

    //duplicate program name for the new thread context
    proc_name = kstrdup(progpath);
    if (proc_name == NULL) {
        return ENOMEM;
    }

    //try to open the executable file
    result = vfs_open(progpath, 0_RDONLY, 0, &vnode);
    if (result) {
        kfree(proc_name);
        return result;
    }

    //allocate a fresh address space for the process
    new_as = as_create();
    if (new_as == NULL) {
        vfs_close(vnode);
        kfree(proc_name);
        return ENOMEM;
    }

    //install and activate the new address space
    prev_as = proc_setas(new_as);
    as_activate();

    //load the ELF binary into memory
    result = load_elf(vnode, entry_out);
    if (result) {
        //on failure, roll back to the previous address space
        vfs_close(vnode);
        proc_setas(prev_as);
        as_activate();
        as_destroy(new_as);
        kfree(proc_name);
    }

```

```

    return result;
}

//file no longer needed once ELF is loaded
vfs_close(vnode);

```

Listing 5.12: Loading the new executable (code from loadexec function)

```

    result = as_define_stack(new_as, stack_out);
if (result) {
    proc_setas(prev_as);
    as_activate();
    as_destroy(new_as);
    kfree(proc_name);
    return result;
}

```

Listing 5.13: Creating a new stack (code from loadexec function)

```

    //discard the previous address space now that we are
    successful
if (prev_as != NULL) {
    as_destroy(prev_as);
}

//update the thread name to reflect the new executable
kfree(curthread->t_name);
curthread->t_name = proc_name;

```

Listing 5.14: Destroying the old address space and updating the thread name (code from loadexec function)

Finally, the process enters the new program context using `enter_new_process`, which transfers control to the new executable. On success, this function does not return.

```

    enter_new_process(argc, user_argv, NULL, stackptr, entrypoint)
;
    panic("sys_execv: enter_new_process returned unexpectedly\n");

    return EINVAL;

```

Listing 5.15: Entering a new process

Error Handling

Condition	Error
Memory allocation failure	ENOMEM
Invalid user pointer	EFAULT
Argument buffer too large	E2BIG
Invalid argument	EINVAL
Executable load failure	ENOEXEC

Chapter 6

Results and Validation

6.1 Overview

In this chapter, we present the validation results obtained from executing various test programs provided within the OS/161 distribution. These tests were essential for verifying the correctness of our system call implementations, ensuring robust error handling, and confirming the stability of the system under concurrent loads.

To support the concurrent execution of multiple processes and the overhead of the new kernel structures, we increased the physical memory available to the system. Specifically, the `ramsize` parameter in `sys161.conf` was increased from the default **512 KB** to **2048 KB** (2 MB). This adjustment prevents `ENOMEM` errors during the creation of new address spaces for forked processes.

6.2 System Validation

6.2.1 Process Management: `forktest`

The `testbin/forktest` program is the primary benchmark for verifying the `fork()` system call. It creates multiple child processes that run concurrently, printing output to the console. Successful execution (Figure 6.1) demonstrates that the process control block (PCB) is correctly copied and that parent/child processes execute independently.

6.3 Error Handling: The `badcall` Suite

The most extensive validation was performed using `testbin/badcall`. This suite deliberately invokes system calls with invalid parameters (e.g., invalid pointers, negative file descriptors) to ensure the kernel returns the correct `errno` codes.

```
OS/161$ forktest
forktest: Starting. Expect this many:
|-----|
AABBBBCCCCCCCCDDDDDDDDDDDDDDDD
forktest: Complete.
(program name unknown): subprocess time: 0.964875888 seconds
```

Figure 6.1: Output of `testbin/forktest` showing concurrent parent/child execution.

Figure 6.2 shows the menu of available tests within the suite.

```
OS/161$ badcall
[a-|, 1-4, *, ?=menu, !=quit]
[a] execv           [b] waitpid
[c] open            [d] read
[e] write           [f] close
[g] reboot          [h] sbrk
[i] ioctl           [j] lseek
[k] fsync            [l] ftruncate
[m] fstat            [n] remove
[o] rename           [p] link
[q] mkdir            [r] rmdir
[s] chdir            [t] getdirentry
[u] symlink           [v] readlink
[w] dup2             [x] pipe
[y] __time           [z] __getcwd
[{}] stat            [!] lstat
[1] asst1            [2] asst2
[3] asst3            [4] asst4
[*] all              [!] quit
```

Figure 6.2: The `badcall` test menu.

6.3.1 Process Management Tests

We validated the robustness of process-related calls, specifically `execv` and `waitpid`. As shown in Figure 6.3, the system correctly identifies invalid pointers, null arguments, and invalid PIDs.

6.3.2 File System Tests

The following figures demonstrate the system's ability to handle edge cases in file I/O operations. Figure 6.4 covers the fundamental operations (`open`, `close`, `read`, `write`), verifying that closed or invalid file descriptors are trapped correctly.

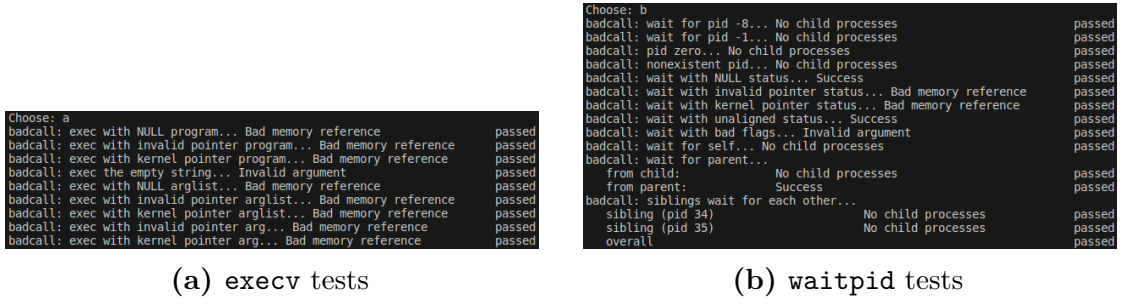


Figure 6.3: Validation of process management error handling.

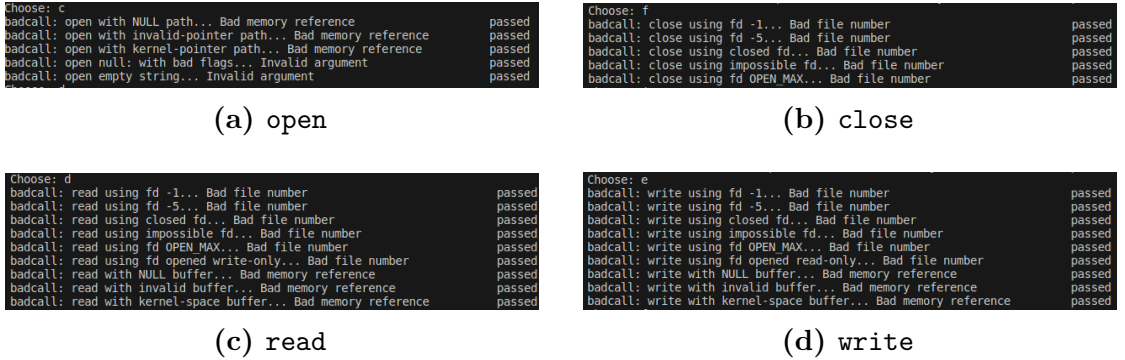


Figure 6.4: Basic File I/O `badcall` results.

Figure 6.5 validates more complex operations: `lseek` (seeking to negative offsets), `dup2` (invalid file descriptors), and `getcwd` (buffer handling).

```
Choose: j
badcall: lseek using fd -1... Bad file number      passed
badcall: lseek using fd -5... Bad file number      passed
badcall: lseek using closed fd... Bad file number  passed
badcall: lseek using impossible fd... Bad file number passed
badcall: lseek using fd OPEN_MAX... Bad file number passed
badcall: lseek on device... Illegal seek          passed
badcall: lseek stdin when open on file...         -----
badcall: try 1: SEEK SET... Success                passed
badcall: try 2: SEEK END... Success                passed
badcall: lseek to negative offset... Invalid argument passed
badcall: seek past/to EOF...                      passed
badcall: lseek with invalid whence code... Invalid argument passed
```

(a) lseek

```
Choose: w
badcall: dup2 using fd -1... Bad file number      passed
badcall: dup2 using fd -5... Bad file number      passed
badcall: dup2 using closed fd... Bad file number  passed
badcall: dup2 using impossible fd... Bad file number passed
badcall: dup2 using fd OPEN_MAX... Bad file number passed
badcall: dup2 to -1... Bad file number            passed
badcall: dup2 to -5... Bad file number            passed
badcall: dup2 to impossible fd... Bad file number passed
badcall: dup2 to OPEN_MAX... Bad file number      passed
badcall: copying stdin to test with... badcall: dup2 to same fd... passed
badcall: fstat fd after dup2 to itself... Success passed
```

(b) dup2

```
Choose: z
badcall: getcwd with NULL buffer... Bad memory reference passed
badcall: getcwd with invalid buffer... Bad memory reference passed
badcall: getcwd with kernel-space buffer... Bad memory reference passed
```

(c) getcwd

Figure 6.5: Advanced File System badcall results.

6.4 Shell and Interactive Testing

Finally, the ultimate integration test was the execution of the system shell (`/bin/sh`). The shell relies on the correct interaction of almost all implemented system calls: `fork` to create process groups, `execv` to run commands, `waitpid` to manage foreground jobs, and file syscalls for redirection.

We verified that the shell can successfully launch commands, read files (e.g., `cat`), and handle input/output redirection, demonstrating a fully functional user-space environment.

```
OS/161$ cat testscripts/test.py
#!/usr/bin/env
# test.py - run some test material
# usage: auto/test.py [options] test-commands
# options:
#   --menuprompt=STR    Change menu prompt string
#   --shellprompt=STR   Change shell prompt string
#   --conf=sys161.conf  Use alternate sys161 config
#   --ram=N             Force RAM size (default from sys161 config)
#   --cpus=N            Force number of cpus (default from sys161 config)
#   --doom=N            Set doom counter to N (default none)
#   --progress=N        Progress monitoring with N-second timeout (default 30)
#   --no-progress       Disable progress monitoring
#   --timeout=N         Global timeout, in seconds (default 300)
#   --kernel=KERNEL     Choose kernel to run (default "kernel")
#
# This is a directly executable wrapper around runtest.py. You can use
# it from the host OS shell to run more or less arbitrary scripts, or
# you can write your own Python scripts using runtest.py directly.
#
# See the top of runtest.py for an explanation of the arguments.
#
import sys
from optparse import OptionParser
import runtest
```

Figure 6.6: Execution of the shell and reading a file via `cat`.